

1. What is Kubernetes (K8s)?

Kubernetes (often abbreviated *K8s*, for the 8 letters between “K” and “s”) is an **open-source container orchestration system** that automatically **deploys, manages, scales, and heals** containerized applications.

It was developed by **Google**, now maintained by the **Cloud Native Computing Foundation (CNCF)**.

Why it exists:

Once you start running dozens or hundreds of containers (e.g., ML APIs, dashboards, data preprocessors), manually starting/stopping them isn't feasible. Kubernetes automates this entire process.

2. Kubernetes Core Purpose

Kubernetes provides:

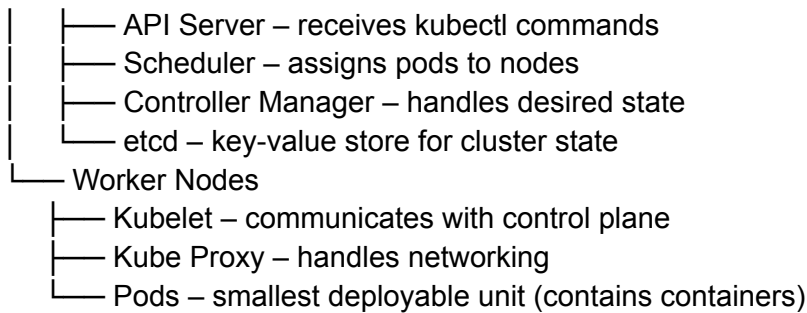
Function	Description
Orchestration	Schedule containers across nodes (machines).
Scaling	Automatically add/remove containers when load changes.
Load Balancing	Distribute traffic evenly between containers.
Self-Healing	Restart failed containers automatically.
Configuration & Secrets Management	Manage environment variables and credentials securely.
Rolling Updates	Deploy new versions gradually without downtime.

→ In short: **Kubernetes = container manager for reliability, scalability, and automation.**

3. Kubernetes Architecture Overview

Cluster

└─ Control Plane



Pod: A group of one or more containers that share storage, network, and configuration.

Node: A physical or virtual machine that runs Pods.

Cluster: A set of nodes managed by a control plane.

4. Why Kubernetes in MLOps?

- **Model serving** — deploy model containers and auto-scale based on traffic.
 - **Batch jobs** — schedule distributed training tasks.
 - **Multi-environment deployment** — dev, staging, production.
 - **Auto-healing** — restart failed ML services.
 - **Horizontal Pod Autoscaler (HPA)** — adjusts replicas based on CPU/memory load.
 - **Integrates with CI/CD** — pipelines deploy containers directly to clusters.
-

5. Scaling in Kubernetes

There are **two types of scaling**:

♦ A. Horizontal Scaling (Scale Out / Scale In)

- Adds or removes **pods** (container replicas).
- Example: If CPU > 80%, create new pod instances.

- Fast and efficient for stateless microservices (like ML APIs).
- Done automatically via **Horizontal Pod Autoscaler (HPA)**.

Before: 2 pods → Load increases → After: 4 pods

✅ **Kubernetes term:** *replicas*

♦ B. Vertical Scaling (Scale Up / Scale Down)

- Increases or decreases **resources (CPU, RAM)** allocated to each pod.
- Useful when workloads require more memory or computation.
- More expensive and slower (may restart pods).

Before: Pod = 1 CPU, 1GB RAM → After: 2 CPU, 4GB RAM

💡 Comparison

Scaling Type	Increases	Example	Best For
Horizontal	Number of Pods	2 → 6 replicas	Web servers, ML inference APIs
Vertical	Resources per Pod	2GB → 8GB RAM	Training jobs, data preprocessing

⚡ 6. Real-life MLOps Example

Suppose you deployed a model API on Kubernetes.

- Morning = few requests → 2 replicas.
- Noon = heavy traffic → HPA adds 4 more replicas (horizontal scale).
- A large training job later → need more CPU → vertical scale that pod's limits.
Kubernetes ensures reliability and cost efficiency.

7. Horizontal vs Vertical Scaling Diagram (textual)

HORIZONTAL:

```
      +-----+      +-----+
Load → | POD 1 | + → + | POD 2 | + → + | POD 3 | ...
      +-----+      +-----+
(Replication)
```

VERTICAL:

```
      +-----+
      | POD 1 |
      +-----+
        ↑
More CPU, RAM assigned
```

8. Quick Recap

- Kubernetes = **container orchestrator** for automation & scaling.
- **Pod** = smallest deployable unit.
- **Horizontal scaling** → adds more pods.
- **Vertical scaling** → increases pod resources.
- Kubernetes ensures **auto-healing**, **load balancing**, and **zero-downtime updates**.

15 MCQs (with answers + explanations)

① **Kubernetes was originally developed by:**

A) AWS B) Microsoft C) Google D) Red Hat

✓ **C.** By Google, now under CNCF.

② **The smallest deployable unit in Kubernetes is:**

A) Container B) Node C) Pod D) Deployment

✓ **C.** Pods group one or more containers.

3) Which component stores cluster state in Kubernetes?

A) API Server B) etcd C) Scheduler D) Kubelet

✓ **B.** etcd is a distributed key-value store.

4) Which command-line tool interacts with the cluster?

✓ **kubectl**

5) Horizontal scaling means:

A) Increasing CPU of same pod

B) Increasing number of pods

✓ **B.** Adds replicas (pods).

6) Vertical scaling means:

✓ Increasing CPU/RAM of existing pods.

7) Which Kubernetes component runs on every node and communicates with the control plane?

✓ **Kubelet.**

8) What handles traffic routing and load balancing between pods?

✓ **Kube Proxy.**

9) What is the function of Scheduler?

✓ Assigns pods to appropriate nodes based on available resources.

10) Which feature restarts failed pods automatically?

✓ **Self-healing** (managed by Controller Manager).

11) HPA adjusts pods based on:

✓ Metrics like CPU, memory, or custom metrics.

12) Which scaling type is faster and stateless-service friendly?

✓ **Horizontal scaling.**

13) Which is more suitable for long ML training jobs?

✓ **Vertical scaling** (needs more memory/CPU per pod).

14) To handle sudden spikes in user requests, which scaling should be used?

✓ **Horizontal scaling.**

15) What best describes Kubernetes?

✓ **An open-source system for automating deployment, scaling, and management of containerized applications.**

10 Short Answer Questions (2–3 sentences each)

1 What is Kubernetes?

A container orchestration platform that automates deployment, scaling, and management of containers across multiple machines.

2 Define a Pod.

The smallest deployable unit in Kubernetes containing one or more tightly coupled containers sharing the same network and storage.

3 What is the role of the Scheduler?

It assigns pods to suitable nodes based on resource requirements and availability.

4 What is Horizontal Pod Autoscaler (HPA)?

A feature that automatically increases or decreases the number of pods based on observed metrics like CPU or memory usage.

5 What is the difference between Horizontal and Vertical Scaling?

Horizontal adds more pods (replicas); vertical increases resources of existing pods.

6 Why is horizontal scaling preferred for APIs?

It allows serving more user requests simultaneously with minimal downtime.

7 When should you use vertical scaling?

When workloads are memory or CPU heavy and can't be parallelized easily (e.g., model training).

8 What ensures Kubernetes automatically restarts failed containers?

The Controller Manager continuously reconciles actual vs desired state and restarts failed pods.

9 How does Kubernetes achieve load balancing?

Through the Service and Kube Proxy components that route traffic evenly across pods.

10 Why is Kubernetes crucial in MLOps?

It ensures ML models and services are scalable, fault-tolerant, and automatically maintained across environments.